

The design, from first principles — why the CanSleep pass is built the way it is

A walk down the reasoning behind the pass. Each step is forced by the one before it: the engineering choices fall out of the analysis theory, not the other way round. Follow the numbers.

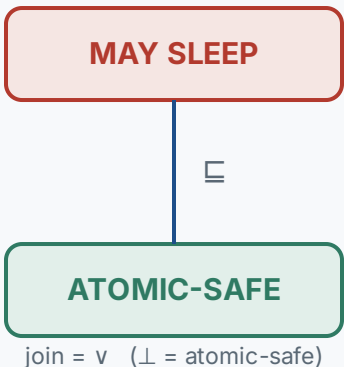
1 The problem fixes the question.

In the kernel some functions **may sleep** — they can call `schedule()` and yield the CPU. Calling one from **atomic context** (an interrupt handler, or while holding a spinlock) can deadlock the machine. To catch that statically, we need exactly one fact about every function: can it sleep?

↓ so the target is a per-function predicate — can this function sleep?

2 That question is reachability.

A function may sleep exactly when **some path through it reaches a blocking primitive**. So the value is a two-point lattice — atomic-safe below, may-sleep above — and a function's value is the **join** (logical or) of its own effect and its callees'.

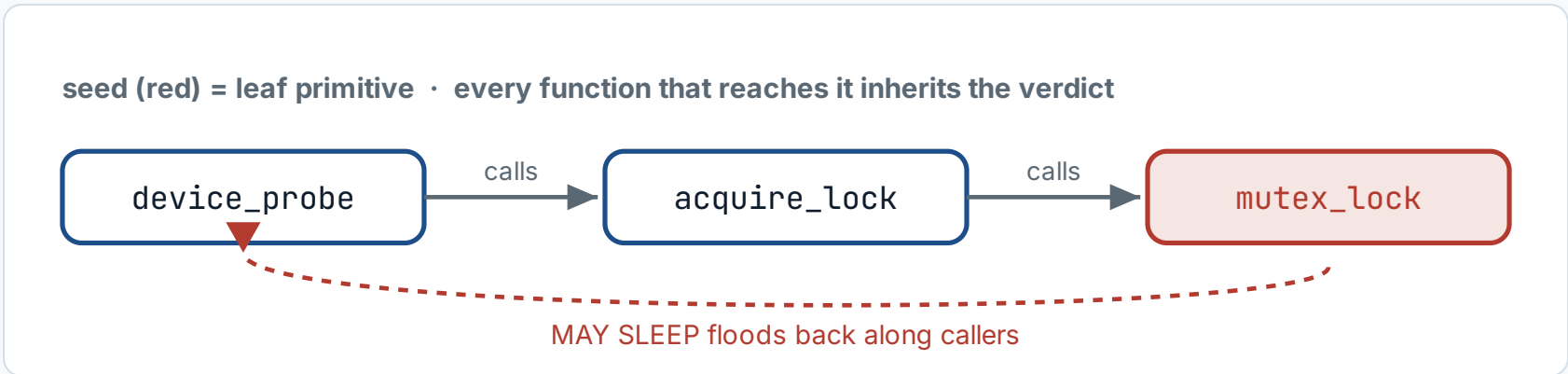


```
CanSleep(f) = directlyBlocks(f) ∨ ∃ g ∈ callees(f). CanSleep(g)
```

↓ seeded at the primitives, that reachability is a least fixpoint.

3 Reachability over the call graph is a least fixpoint.

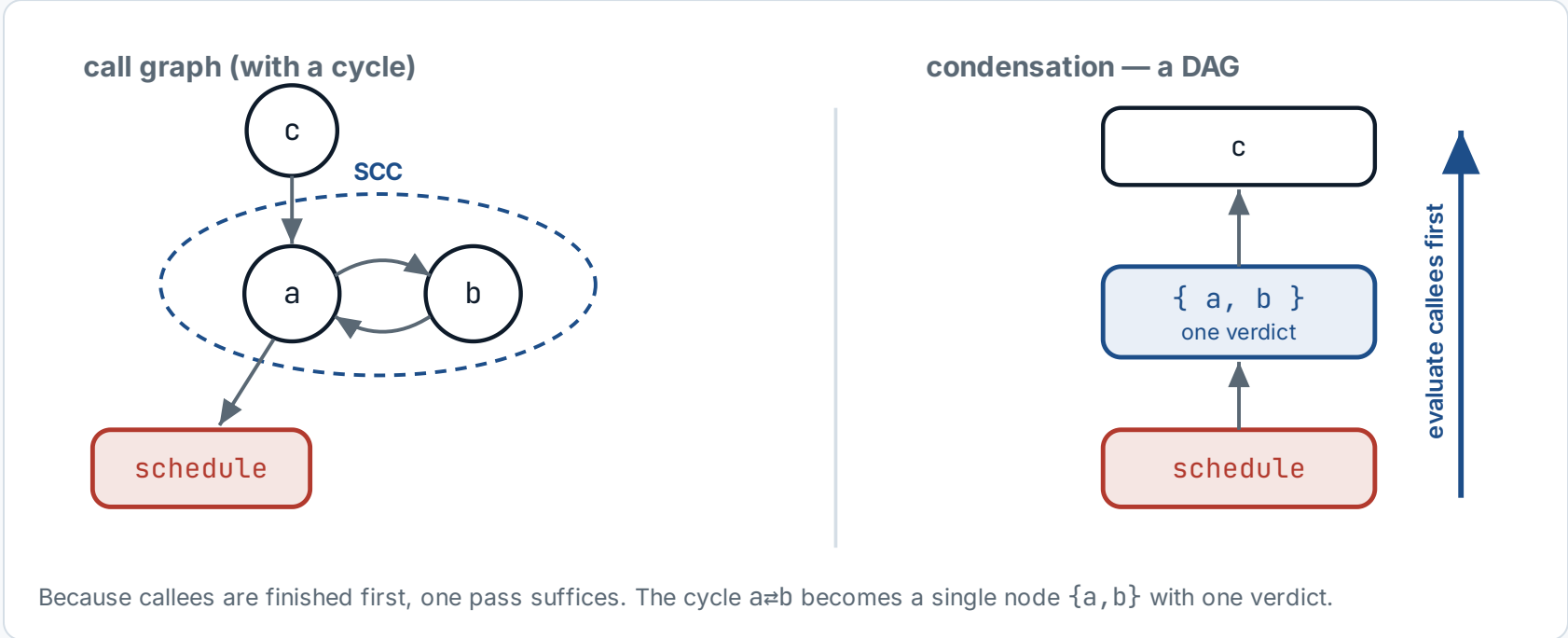
Seed the may-sleep set with the leaf blocking primitives — `mutex_lock`, `schedule`, `msleep`, `might_sleep` — then close it under the caller-of relation until nothing changes. A may-sleep function is any function that can reach a seed.



↓ computed naïvely this re-sweeps until stable — but the work can be ordered so it needn't.

4 Order by SCCs and one bottom-up sweep is exact.

Collapse each cycle into a strongly connected component; the condensation is a DAG. Visit it in **reverse-topological order** — LLVM's `scc_iterator` yields exactly this — so every callee is finalised before its callers. Then a **single pass** computes the fixpoint with no iteration, and each SCC collapses to one shared verdict.



Why one sweep is exact soundness

Every sleep originates at a leaf primitive, which is an external declaration — never a defined member of an SCC. So any sleeping path either ends at a primitive some member calls **directly**, or exits to a **lower, already-finished** SCC. It cannot leave and return, or the target would sit in the same SCC. Checking both cases over all members captures every path. ■

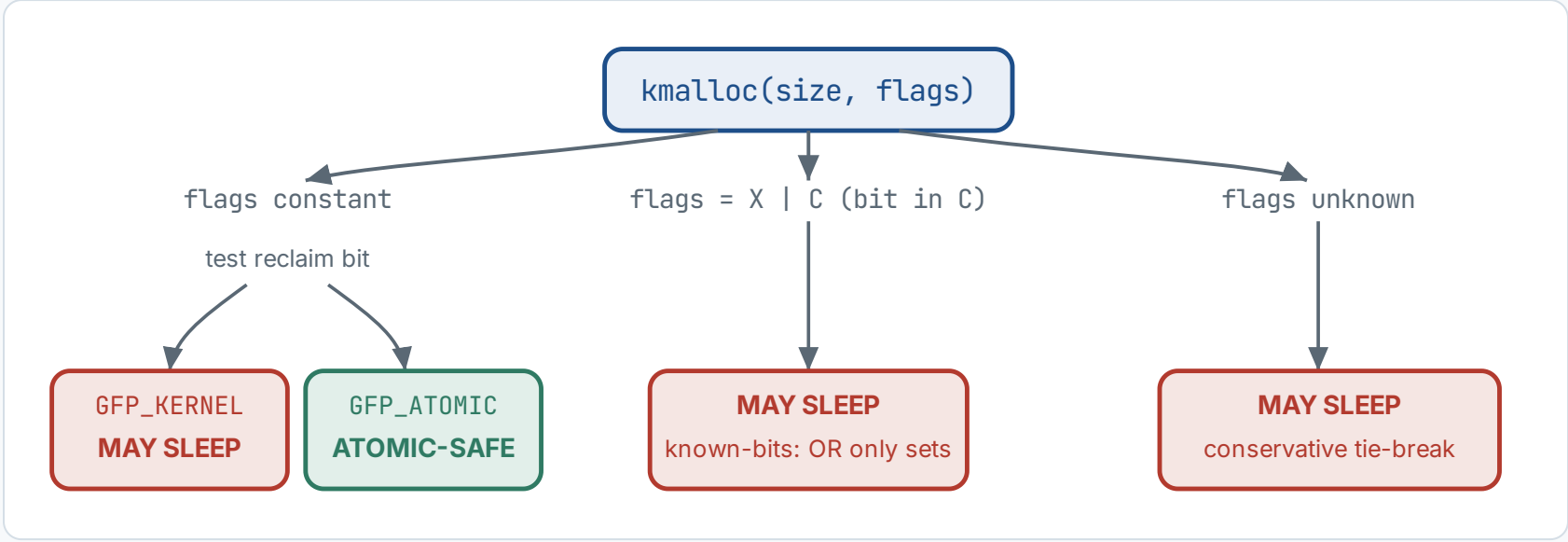
Recursion is free no special case

A recursive cycle is just a non-trivial SCC. Its members are mutually reachable, so they must share one verdict — which is exactly what collapsing the SCC gives. Nothing extra to write.

↓ a name decides most primitives — but not the allocator.

5 One primitive needs its arguments, not just its name.

`kmalloc` sleeps or not depending on its gfp flags, so matching it by name alone would be wrong. The pass reads the flags and decides on the `__GFP_DIRECT_RECLAIM` bit: set means it may reclaim and sleep; clear means it cannot.



↓ and where the flags can't be proven, soundness sets the tie-break — which draws the boundary.

6 Where it can't prove, it says so — and stops there.

The same soundness rule that resolves unknown flags also fixes the honest edges of the tool. Each limit below is a deliberate scope line, not an accident.

- **Unknown flags → may sleep.** A sound over-approximation; never a silent miss.
- **Indirect calls skipped.** No static callee to follow without points-to info.
- **No cross-call flag threading.** Only intraprocedural known-bits are resolved.
- **Context-insensitive.** One verdict per function, independent of call site.
- **C-side of CLSC only.** Rust typestate and the FFI join are future work.
- **Reclaim bit modelled.** Hard-coded here; read from real gfp headers in production.